

ATLAS GYM MANAGEMENT SYSTEM

Informe Técnico: Módulo de Avisos y Notificaciones

Desarrollo por fases de alta rigurosidad · 100% Tests Vitest · React Doctor 100/100 · Rama: `ramaLucas`

FASES 1, 2 Y 3 COMPLETADAS

NEXT.JS TURBOPACK

0 WARNINGS DE CALIDAD

19 / 19

TESTS UNITARIOS PASADOS
(VITEST)

100 / 100

AUDITORÍA REACT DOCTOR (0
ISSUES)

10 / 10

RUTAS EN PRODUCCIÓN (NEXT
BUILD)

1. Contexto y Requerimiento del Cliente

El gimnasio requería una vía directa y confiable para que la administración pudiera comunicarse con los alumnos (avisos de feriados, modificaciones horarias, incorporación de nuevo equipamiento y recordatorios de cuotas vencidas/pendientes), resolviendo a la vez el problema de que la campana de notificaciones del encabezado estaba desvinculada y redirigía erróneamente a Finanzas.

Se acordó e implementó una **metodología de trabajo en 4 fases estrictas**, en la cual cada fase fue verificada con la batería completa de testing antes de solicitar aprobación del usuario para avanzar.

2. Arquitectura de la Solución

- **Capa de Datos Centralizada:** Tipado estricto (`Aviso` , `CategoriaAviso`) en `lib/types.ts` , con estilos temáticos unificados (`CATEGORIA_AVISO_STYLES`).
- **Custom Hook React Doctor Compliance:** `lib/use-avisos.ts` maneja el estado de avisos con updaters 100% puros y sincronización asíncrona mediante `useEffect` hacia `localStorage` , evitando violaciones de renderizado.
- **Navegación Omnicanal:** Integración de la campana interactiva en Header de Alumno y Header de Admin, enlaces en Sidebar de escritorio y barra móvil (`MobileNav`) con badges numéricos reactivos.
- **Canal de Consulta Inmediata:** Integración nativa con WhatsApp oficial de Atlas Gym (`+54 9 261 566-5067`) precargando el asunto del comunicado para una atención rápida al alumno.

3. Fases Ejecutadas y Cambios de Código

Fase 1: Capa de Datos, Store y Pruebas Unitarias

Se construyó la infraestructura base de datos y se blindó con una suite de pruebas en Vitest.

Archivo	Tipo	Descripción del Cambio
lib/types.ts	Modificado	Definición de interfaz <code>Aviso</code> , categorías <code>IMPORTANTE</code> , <code>NOVEDAD</code> , <code>HORARIO</code> y diccionarios de estilos y badges.
lib/mock-data.ts	Modificado	Creación de colección <code>AVISOS MOCK</code> con comunicados generales y privados.
lib/use-avisos.ts	Nuevo	Hook desacoplado para gestión de estado, lecturas, conteos y persistencia local sin efectos colaterales impuros.
lib/store.tsx	Modificado	Exposición en el contexto global <code>useAppData()</code> : <code>avisos</code> , <code>crearAviso</code> , <code>marcarAvisoLeido</code> , <code>eliminarAviso</code> , <code>getAvisosParaUsuario</code> y <code>getCantidadAvisosNoLeidos</code> .
lib/avisos.test.ts	Nuevo	9 tests unitarios para verificar visibilidad por rol, conteo de no leídos y marcado idempotente de lectura.
vitest.config.ts	Modificado	Configuración de filtros de inclusión/exclusión para testing ágil excluyendo <code>node_modules</code> .

Fase 2: Campana de Notificaciones Interactiva en Header, Sidebar y Mobile

Se conectó la campana del header para llevar a `/avisos` , mostrando un badge reactivo con el total de comunicados sin leer tanto en el portal del alumno como en el del administrador.

Archivo	Tipo	Descripción del Cambio
components/sidebar.tsx	Modificado	Nuevo ítem <code>Avisos</code> (icono <code>Bell</code>) con badge numérico en modo expandido y dot en modo colapsado.
components/mobile-nav.tsx	Modificado	Inclusión de <code>Avisos</code> en la barra móvil con badge contador reactivo y atributos de accesibilidad ARIA.
app/page.tsx	Modificado	Subcomponente unificado <code>HeaderStatusSection</code> conectando la campana a <code>/avisos</code> . Nuevo <code>AvisoBannerDestacado</code> en Home Alumno para alertar sobre el aviso más reciente. Refactor <code>AlumnoCardsSection</code> para reducir complejidad ciclomática.

Fase 3: Pantalla Completa `/avisos` (Portal Alumno y Panel Admin)

Se construyó la pantalla completa con vista adaptada para cada rol, modal accesible de creación y buscador con chips.

Archivo	Tipo	Descripción del Cambio
components/modal-nuevo-aviso.tsx	Nuevo	Modal nativo <code><dialog></code> accesible con selector de categoría, selector de destinatario (Todos o Alumno específico), validaciones y feedback de error.
app/avisos/page.tsx	Nuevo	Vista completa con chips de filtrado (Todos, No leídos, Generales, Individuales, Categorías), barra de búsqueda en tiempo real, tarjetas de alumno con botón WhatsApp y marcar como leído, tarjetas de admin con recibo de lectura y eliminación, y estado vacío elegante (Inbox).

4. Comparativa de Código: Antes vs Después

A. Enlace de la Campana del Header en `app/page.tsx`

Antes: La campana redirigía estáticamente a finanzas usando como badge la cantidad de pagos pendientes.

```
<Link href="/finanzas" title="pagos pendientes" >
  <Bell className="size-5" />
  {pagosPendientes.length > 0 && (
    <span>{pagosPendientes.length}</span>
  )}
</Link>
```

Después: Enlace unificado a `/avisos` mostrando reactivamente la cantidad de avisos no leídos del usuario conectado.

```
<Link
  href="/avisos"
  title={cantNoLeidos > 0 ? `${cantNoLeidos} avisos sin leer` : 'Avisos y Comunicados'}
  className="relative text-slate-400 transition-[color,transform] duration-200 hover:text-white"
>
  <Bell className="size-5" />
  {cantNoLeidos > 0 && (
    <span className="absolute -right-1 -top-1 flex size-4 items-center justify-center rounded-full bg-rose-500"
      {cantNoLeidos}
    </span>
  )}
</Link>
```

B. Integración con WhatsApp para Alumnos en `app/avisos/page.tsx`

```
const whatsappMensaje = `${CONTACTO_ATLAS.whatsappUrl}?text=${encodeURIComponent(
  `Hola Atlas Gym! Tengo una consulta sobre el aviso: "${aviso.titulo}"`
)}`

<a
  href={whatsappMensaje}
  target="_blank"
  rel="noopener noreferrer"
  className="inline-flex items-center gap-1.5 rounded-xl bg-emerald-50 px-3.5 py-1.5 text-xs font-bold text-emerald-600"
>
  <MessageCircle className="size-3.5 text-emerald-600" />
  Consultar por WhatsApp
</a>
```

5. Resultados de la Batería de Pruebas Automatizadas

Herramienta de Verificación	Métricas Obtenidas	Evaluación
Vitest Suite (<code>npm test</code>)	19 de 19 pruebas unitarias exitosas (100% pasando).	APROBADO
React Doctor (<code>npm run doctor</code>)	Score 100 / 100 Great · 36 archivos auditados · 0 advertencias · 0 errores.	APROBADO
Next.js Build (<code>npm run build</code>)	Compilación limpia Turbopack · 10 rutas generadas (SSG / SSR) · 0 errores.	APROBADO

6. Registro de Versiones en Git (Rama `ramaLucas`)

- `075ff96` : *feat(avisos): phase 1 - data models, store manager and unit tests*
- `8ce1ed4` : *feat(avisos): phase 2 - notification bell, dynamic badges and navigation links in header, sidebar and mobile nav*
- `50ea827` : *feat(avisos): phase 3 - avisos full page, student cards with whatsapp action, admin publisher modal and filters*